

DataAnalystAgent

Documentazione tecnica del progetto

Un agente conversazionale che combina **retrieval-augmented generation** su documentazione tecnica e **code execution** su dati tabellari, per rispondere autonomamente a domande in linguaggio naturale sul dataset World Development Indicators (WDI) della Banca Mondiale.

Componente	Tecnologia
Orchestrazione agente	LlamaIndex — ReActAgent
Modello linguistico	OpenRouter (modello openrouter/owl-alpha)
Retrieval (RAG semantico)	sentence-transformers (all-MiniLM-L6-v2, locale) + FAISS (IndexFlatIP, in-memory)
Manipolazione dati	pandas, numpy
Dataset	World Development Indicators (WDI), Banca Mondiale — CSV
Esecuzione codice	Sandbox Python con builtins ristretti

Sommario

1. Panoramica del progetto	3
1.1 Perché un'architettura a due tool non basta da sola	3
2. Architettura generale	3
2.1 Componenti principali	4
3. Pipeline di preparazione dei dati	5
3.1 Normalizzazione del CSV principale.....	5
3.2 Chunking e costruzione degli indici semantici	5
3.3 Similarità e filtro per metadata	5
4. Strategia di decisione a due fasi.....	5
Fase 1 — Determinare e verificare l'indicatore.....	5
Fase 2 — Recuperare e calcolare	6
5. I quattro tool dell'agente	7
5.1 inspect_schema(query: str = "").....	7
5.2 retrieve_docs(query: str).....	7
5.3 retrieve_metadata(query, series_code=None, country_code=None).....	7
5.4 run_python(code: str).....	7
6. Ciclo di esecuzione ReAct.....	7
7. Limiti noti (stato attuale)	8
8. Cosa farei meglio con più tempo	8
8.1 Una possibile variante: architettura a 3 agenti.....	8
8.2 Lo sviluppo Spec-Driven Development adottato per mancanza di tempo.....	9
8.3 Altre rifiniture prioritarie	9
8.4 Riepilogo delle scelte di design	9
9. DEMO	10

1. Panoramica del progetto

L'obiettivo del progetto è costruire un agente AI in grado di rispondere a domande in linguaggio naturale su un dataset tabellare di grandi dimensioni (WDI), combinando due capacità complementari:

- **Code execution:** esecuzione di snippet Python in una sandbox per calcoli, filtri e aggregazioni sui dati.
- **Retrieval (RAG):** ricerca su un corpus di documentazione tecnica che descrive il significato, la metodologia e le fonti degli indicatori del dataset.

Per ogni domanda l'agente deve stabilire autonomamente se servono i dati, la documentazione, entrambi, o nessuno dei due.

1.1 Perché un'architettura a due tool non basta da sola

Avere due tool distinti è una condizione necessaria ma non sufficiente. Senza una strategia esplicita di composizione, un agente ReAct generico rischia di:

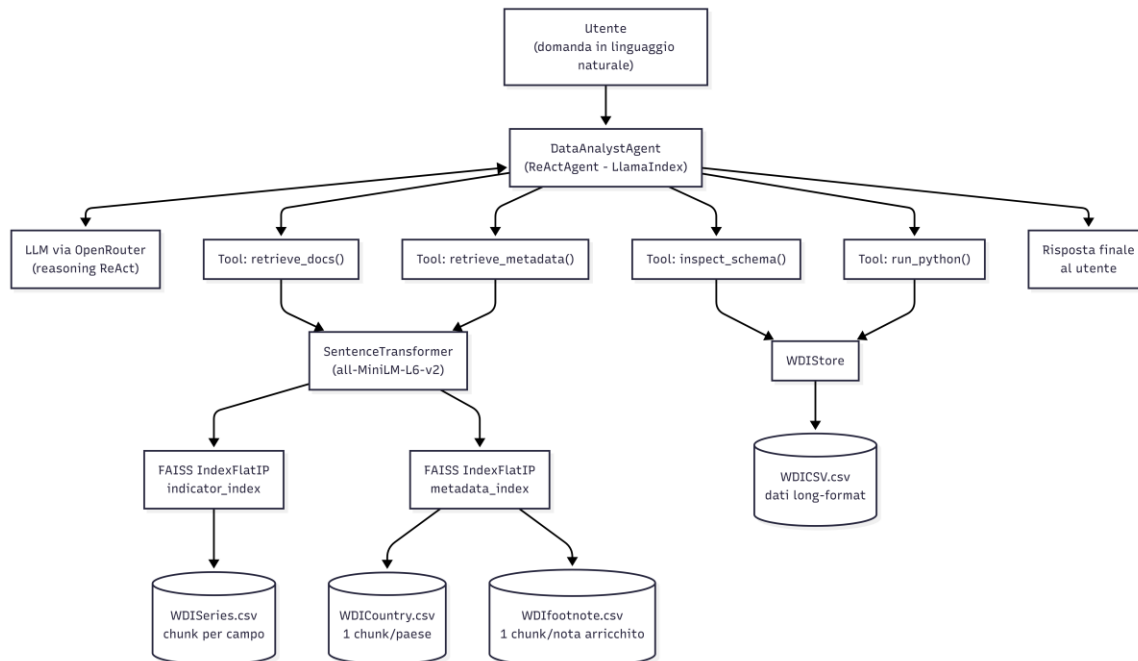
- scegliere il tool sbagliato per mancanza di indicazioni esplicite nel prompt di sistema;
- non avere un modo per collegare l'informazione semantica (nome/definizione di un indicatore) con la sua rappresentazione fisica nei dati (nome colonna, codice esatto nel DataFrame);
- ripetere chiamate di retrieval inutili o scrivere codice 'a tentativi' senza prima verificare lo schema reale dei dati.

Il progetto risolve questo problema introducendo un **quarto tool di raccordo/qualità** (*inspect_schema* per lo schema, *retrieve_metadata* per qualità/comparabilità dei dati) e una **policy esplicita a due fasi** dichiarata nel system prompt dell'agente ReAct, oltre a un retrieval semantico (vedi sezione 3).

2. Architettura generale

Il sistema è composto da tre macro-blocchi: il livello dati (*WDIStore*), il livello di orchestrazione agentica (*DataAnalystAgent* su *ReActAgent* di *LlamaIndex*), e il livello LLM remoto (*OpenRouter*).

Diagramma 1 — Architettura generale



2.1 Componenti principali

Classe / funzione	Responsabilità
WDIStore	Carica i CSV WDI, normalizza il dataset principale (wide → long via melt), costruisce gli indici semantici e le mappe di lookup indicatore.
SemanticIndex	Wrapper generico su FAISS (IndexFlatIP + normalizzazione L2) e SentenceTransformer; build() ed encode/search riutilizzati sia per indicatori sia per metadata.
run_python_sandbox()	Esegue codice Python fornito dall'LLM in un ambiente con builtins ristretti, operando su una copia del DataFrame.
DataAnalystAgent	Inizializza LLM, i 4 tool e il ReActAgent; espone i metodi query()/answer().
SYSTEM_PROMPT	Dichiara esplicitamente la strategia a due fasi, regole di utilizzo dei 4 tool, e soprattutto vieta di suggerire step all'utente — l'agente DEVE eseguire autonomamente.

3. Pipeline di preparazione dei dati

Al momento dell'inizializzazione, **WDIStore** carica cinque file del dataset WDI ed esegue una serie di trasformazioni preparatorie.

3.1 Normalizzazione del CSV principale

Il file **WDICSV.csv** arriva in formato **wide**: una riga per coppia (paese, indicatore), una colonna per ogni anno. Viene trasformato in formato **long** tramite **pandas.melt()**: ogni anno diventa un valore nella colonna **year**, affiancato dalla colonna **value**.

3.2 Chunking e costruzione degli indici semantici

Vengono costruiti **due indici FAISS separati** (**indicator_index** e **metadata_index**), popolati con embedding prodotti da **all-MiniLM-L6-v2** (sentence-transformers, locale). La scelta di tenerli separati è motivata dalla diversa granularità semantica dei corpus, ma la strategia di chunking è esplicita e differenziata per tipo di sorgente:

- **Series (indicatori)**: un chunk per campo logico (**short_definition**, **long_definition**, **methodology**, **development_relevance**), non un unico blob concatenato. Il nome dell'indicatore viene ripetuto in ogni chunk per dargli contesto anche se recuperato isolatamente.
- **Footnote**: un chunk per nota (la granularità naturale del record), ma il testo viene arricchito prima dell'embedding con il contesto esplicito — "Nota sui dati per IT, indicatore ..., anno 2015: ..."
- **Country**: un record = un chunk (paese, regione, fascia di reddito), nessuno splitting ulteriore.

Tenere indicatori e metadata su due indici FAISS distinti evita inoltre che le definizioni — più lunghe — competano nello stesso spazio vettoriale con le footnote, molto più brevi e puntuali.

3.3 Similarità e filtro per metadata

SemanticIndex usa **IndexFlatIP** di FAISS su embedding normalizzati L2, equivalente a cosine similarity. **retrieve_metadata()** accetta opzionalmente **series_code/country_code** per pre-filtrare i candidati sui metadata del chunk prima di restituire i risultati.

4. Strategia di decisione a due fasi

Il cuore del progetto è la policy dichiarata esplicitamente nel **SYSTEM_PROMPT** passato al ReActAgent. Il prompt include una sezione **CRITICAL INSTRUCTIONS** che vieta esplicitamente all'agente di suggerire step all'utente — l'agente è responsabile dell'esecuzione autonoma.

Fase 1 — Determinare e verificare l'indicatore

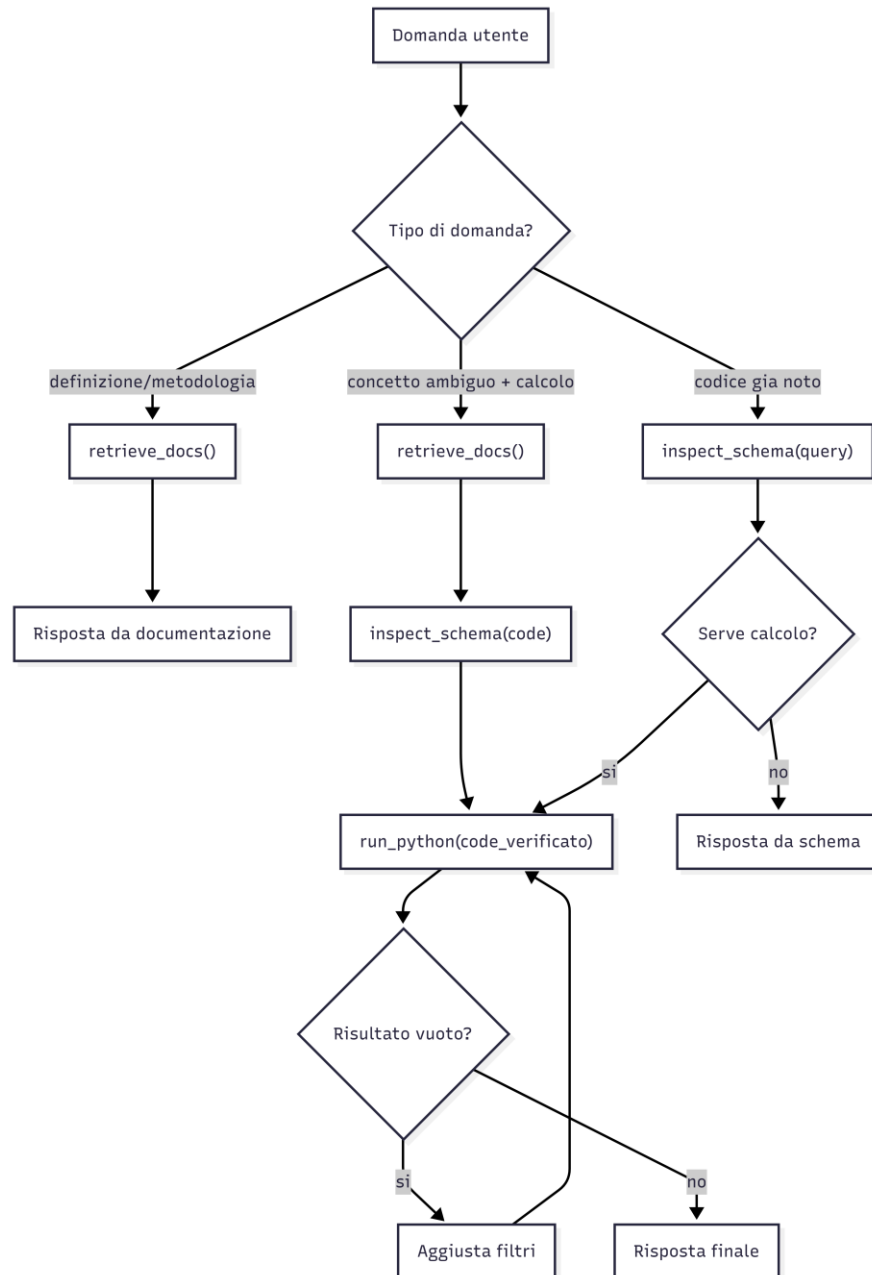
- Domanda su definizione/metodologia/unità → **retrieve_docs()**
- Concetto ambiguo (es. "povertà", "PIL") → prima **retrieve_docs()**, poi **inspect_schema()**
- Domanda su qualità/comparabilità → **retrieve_metadata()**

- Lookup diretto per codice noto → `inspect_schema()`

Fase 2 — Recuperare e calcolare

- Domanda che richiede valori/calcoli → `run_python()` con Series Code verificato
- Se risultato è vuoto/NaN → aggiustare filtri e retry

Diagramma 2 — Albero decisionale a due fasi



5. I quattro tool dell'agente

5.1 inspect_schema(query: str = "")

Tool di raccordo per collegare semantica e schema fisico.

- Senza argomenti: ritorna struttura DataFrame — colonne, dtype, range anni
- Con argomento: esegue lookup esatto/fuzzy su nome o codice indicatore

5.2 retrieve_docs(query: str)

Ricerca semantica sui chunk per-campo dei `WDISeries.csv`. Restituisce i 5 chunk più simili con `indicator_code`, `field_type` e score di similarità.

5.3 retrieve_metadata(query, series_code=None, country_code=None)

Ricerca semantica sull'indice country+footnote, con pre-filtro esatto su identificatori noti. Combina precisione lessicale su codici con similarità semantica su testo.

5.4 run_python(code: str)

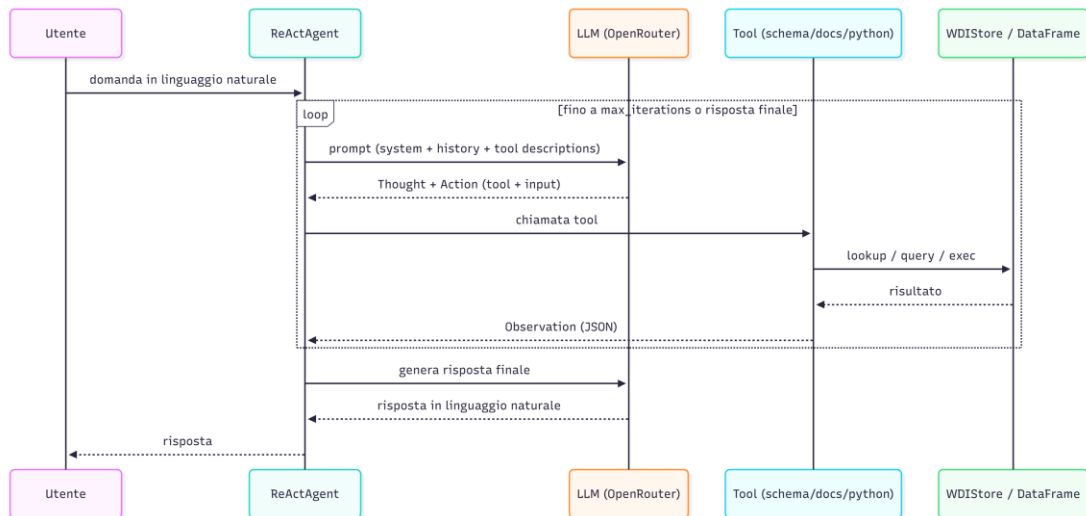
Esegue lo snippet in sandbox con `__builtins__` a whitelist esplicita. Il DataFrame viene passato come copia.

Categoria	Funzioni disponibili
Base	len, min, max, sum, round, sorted, list, dict, float, int, str
Matematica	abs, pow, divmod
Statistica (numpy)	mean, std, median, var
Tipi	isinstance, type, bool
Iterazione	enumerate, zip, map, filter, range

6. Ciclo di esecuzione ReAct

Il `ReActAgent` alterna ragionamento (Thought), scelta ed esecuzione di tool (Action), osservazione (Observation), fino a risposta finale o limite di iterazioni.

Diagramma 3 — Sequenza di esecuzione ReAct



Guardrails applicati:

- **max_iterations** a 8 — più spazio per cicli ReAct complessi su 4 tool
- Sistema prompt con **CRITICAL_INSTRUCTIONS** che vieta suggerimenti all'utente
- Post-processing della risposta che rimuove suggerimenti residui (fallback difensivo)

7. Limiti noti (stato attuale)

- **Context window ridotto** (4096 token): limita cronologia ReAct su cicli complessi
- **Sandbox a livello base**: non disponibili **groupby/merge** per aggregazioni multi-dimensionali
- **Nessuna persistenza degli indici FAISS**: ricostruiti a ogni avvio (encoding incluso)
- **Retrieval puramente denso** (no ibrido): l'embedding è ottimo su sinonimi ma meno preciso su codici esatti (SI.POV.DDAY)
- **Matching fuzzy semplice**: substring case-insensitive, non gestisce sinonimi o errori di battitura

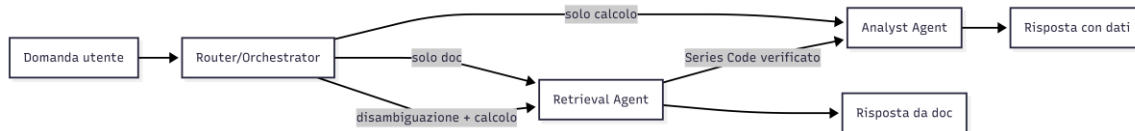
8. Cosa farei meglio con più tempo

8.1 Una possibile variante: architettura a 3 agenti

Oggi la strategia a due fasi è **implicita** nel system prompt di un singolo ReActAgent. Un'alternativa è renderla **esplicita a livello architetturale** con tre agenti specializzati:

- **Router/Orchestrator**: decide deterministicamente se serve fase 1, fase 2, o entrambe
- **Retrieval Agent**: vede solo **retrieve_docs**, **retrieve_metadata**, **inspect_schema** — disambigua l'indicatore
- **Analyst Agent**: vede solo **run_python** — calcola sui dati verificati

Diagramma 3 — Variante a 3 agenti



Valutazione: I vantaggi reali ci sono (prompt più piccoli, testing in isolamento, modelli diversi per ruolo), ma per la scala attuale del progetto (un dataset, 4 tool, domande tipicamente 2-3 step) la complessità (più chiamate LLM, latenza, schema di passaggio messaggi) probabilmente **supera il beneficio**. La consiglierei come passo successivo motivato da dati empirici, non come scelta a priori.

8.2 Lo sviluppo Spec-Driven Development adottato per mancanza di tempo

L'approccio SDD (Specification Driven Development) nel mio progetto è partito dalla definizione di un prompt iniziale in cui ho descritto la struttura e gli obiettivi dell'applicazione.

Da questo prompt è stato generato un file Markdown che formalizza in modo dettagliato l'architettura del progetto e tutti gli step necessari alla sua realizzazione.

Questo documento funge da specifica centrale, utilizzata come riferimento unico e strutturato per guidare l'assistente di coding nella generazione del codice.

È stata una scelta pragmatica dettata dal tempo a disposizione, non quella che avrei fatto con un progetto più lungo. Con più tempo, l'approccio SDD sarebbe stato meno utilizzato e sarebbe stato applicato in modo più rigoroso, cioè:

- Scrivere prima una **specifica formale** (requisiti funzionali, contratti di tool, criteri di accettazione) come documento versionato.
- Derivare una **suite di test** (unit test, test di integrazione, set di domande di valutazione end-to-end con risposta attesa)

In sintesi: l'approccio ha permesso di iterare rapidamente sulle decisioni architetturali (TF-IDF → chunking semantico → FAISS), ma andrebbe chiuso da un ciclo di verifica più formale prima di considerare il codice production-ready.

8.3 Altre rifiniture prioritarie

- **Persistenza degli indici FAISS:** salvare `index.faiss` + metadata su disco, evitare ricalcolo embedding a ogni avvio
- **Retrieval ibrido:** affiancare BM25 all'embedding (con re-ranking) per precisione su codici/acronimi esatti
- **Ampliare la sandbox** a `groupby/merge`, su evidenza di fallimenti reali
- **Logging strutturato** delle chiamate tool (query, tool scelto, esito, latenza) per dataset di valutazione reale
- **Aumentare il context window** del modello LLM, oggi vincolo stretto per cicli ReAct con 4 tool

8.4 Riepilogo delle scelte di design

Decisione	Motivazione sintetica
Indici FAISS separati	Granularità semantica diversa; evita che definizioni

	lunghe competano con footnote brevi
Chunking per campo (series)	Una long_definition mescolata a short_definition annacqua il segnale embedding
Tool inspect_schema()	Collega semantica (retrieval) e schema fisico (colonne/codici DataFrame)
retrieve_metadata() con pre-filtro	Un codice come SI.POV.DDAY non ha significato semantico: filtro esatto è più affidabile
Strategia nel system prompt	La policy di orchestrazione vive in un unico posto coerente
Builtins sandbox a whitelist	Si parte dal livello minimo, espandendo su evidenza di fallimento reale
3 agenti valutati ma non adottati	Beneficio reale solo a scala maggiore; complessità non giustificata oggi
CRITICAL_INSTRUCTIONS nel prompt	Vieta esplicitamente di suggerire step all'utente — agente esegue autonomamente

9. DEMO

